Database Systems Lab — Week 04 Manual

JOINs & Relational Data Modelling

Course: Database Systems Lab (Data Science Program)

Duration: 3 hours (2 lab sessions)

By the end of this lab you will be able to:

- Explain why relational databases split data across multiple tables
- Write INNER, LEFT, RIGHT, and FULL OUTER joins correctly
- Understand what foreign keys are and why they matter
- Combine joins with the aggregation and window function skills from Week 3
- Read and sketch a basic ER diagram
- Recognise the difference between 1NF, 2NF, and 3NF
- Commit multi-table query scripts to GitHub

Why This Matters

You have been working with a multi-table database since Week 2 without fully understanding why the data is split the way it is. This week you find out.

Every time you write a query against a single table you are working with incomplete information. The orders table tells you an order exists and what it cost, but not who placed it or what was in it. That information deliberately lives somewhere else. JOINs are how you bring it back together.

Understanding why data is split, and how to reconnect it, is the difference between someone who can run a query someone else wrote and someone who can design a database, build a pipeline, and answer questions that nobody has thought to ask yet. Every project in Phase 2 of this course will require this skill. Every data engineering role in industry requires it daily.

Part 1 — Why Tables Are Split: Relational Design

Before touching any SQL, you need to understand the problem that relational databases are solving.

Imagine you stored everything in one table:

order_id	customer_name	customer_email	customer_city	product_name	category	price	quantity
1	Ali Hassan	ali@gmail.com	Peshawar	Wireless Mouse	Electronics	1850	1
1	Ali Hassan	ali@gmail.com	Peshawar	Python Book	Books	1200	1
2	Sara Khan	sara@yahoo.com	Lahore	Keyboard	Electronics	7500	1

Three problems appear immediately.

Redundancy. Ali's name, email, and city are repeated on every row of every order he ever places. A customer who places 20 orders has their details stored 20 times.

Update anomalies. If Ali moves from Peshawar to Islamabad, you have to update every single row that mentions him. Miss one and your data is inconsistent.

Deletion anomalies. If you delete the only order containing a certain product, you lose all information about that product.

The solution is to split the data into separate tables — one for customers, one for products, one for orders — and connect them using keys. This is called normalisation, and you have been living with the results of it since Week 2 without knowing it.

Foreign Keys

A foreign key is a column in one table that refers to the primary key of another table. It is the mechanism that connects tables together.

In your schema:

```
-- The orders table has a foreign key pointing to customers
orders.customer_id  →  customers.customer_id

-- The order_items table has foreign keys pointing to two tables
order_items.order_id    →  orders.order_id
order_items.product_id  →  products.product_id
```

Foreign keys enforce referential integrity — they prevent you from creating an order for a customer that does not exist, or adding an order item for a product that has been deleted. The database itself protects your data from becoming inconsistent.

Run this in psql and read the output carefully:

```
-- See the table structure including foreign key constraints
\d orders
\d order_items
```

You will see lines labeled "Foreign-key constraints" that show exactly which columns in which tables are linked together. This is the map of your schema's relationships.

The ER Diagram

An Entity-Relationship (ER) diagram is a visual map of the tables in your database and the relationships between them. You do not need drawing software — understanding how to read one is what matters right now.

Your current schema looks like this:

```
customers           orders              order_items         products
__________          __________          ___________         __________

customer_id (PK)◄──customer_id (FK)    item_id (PK)        product_id (PK)
name               order_id (PK)◄─────  order_id (FK)     ──►product_id
email              order_date           product_id (FK)─┘   product_name
city               status               quantity             category
signup_date        total_amount         unit_price           price
                   shipped_date                              stock_qty
```

The arrows show foreign key relationships. Read them as "each order belongs to one customer" and "each order_item belongs to one order and one product." This is called a one-to-many relationship — one customer can have many orders, one order can have many order_items.

Spend two minutes looking at this before moving on. Every JOIN you write this week is navigating one of these arrows.

Part 2 — Core Concepts

2.1 INNER JOIN

An INNER JOIN returns only the rows where the join condition matches in both tables. If a row in the left table has no match in the right table, it does not appear in the result. Same for the right table.

```
-- The most common join: connect orders to the customers who placed them
SELECT o.order_id,
       c.name          AS customer_name,
       c.city,
       o.order_date,
       o.total_amount,
       o.status
FROM orders o
INNER JOIN customers c ON o.customer_id = c.customer_id
ORDER BY o.order_date DESC;
```

Notice the aliases: `orders o` and `customers c`. When joining tables you almost always use short aliases because you need to prefix column names to avoid ambiguity — both tables might have a column called id, and PostgreSQL would not know which one you mean without the prefix.

The `ON` clause is where you specify the relationship. It is saying: "Match each row in orders to the row in customers where the customer_id values are equal."

```
-- Three-table join: bring in product details alongside order items
SELECT o.order_id,
       c.name          AS customer_name,
       p.product_name,
       p.category,
       oi.quantity,
       oi.unit_price,
       oi.quantity * oi.unit_price  AS line_total
FROM order_items oi
INNER JOIN orders   o ON oi.order_id   = o.order_id
INNER JOIN customers c ON o.customer_id  = c.customer_id
INNER JOIN products  p ON oi.product_id  = p.product_id
ORDER BY o.order_id, p.category;
```

This is the full picture of a transaction: who bought it, what they bought, how many, and at what price. This is the kind of query that feeds a sales report or a recommendation model's training data.

2.2 LEFT JOIN

A LEFT JOIN returns all rows from the left table, plus any matching rows from the right table. Where there is no match, the right-side columns come back as NULL.

```
-- Find ALL customers, including those who have never placed an order
SELECT c.customer_id,
       c.name,
       c.city,
       c.signup_date,
       COUNT(o.order_id)        AS total_orders,
       COALESCE(SUM(o.total_amount), 0) AS total_spent
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name, c.city, c.signup_date
ORDER BY total_spent DESC;
```

If you used an INNER JOIN here, customers who have never ordered would simply disappear from the results. With LEFT JOIN they appear with a 0 for total_spent and 0 for total_orders. In data science this distinction matters enormously — a customer who signed up but never bought anything is a completely different data point from one who never existed.

2.3 RIGHT JOIN

A RIGHT JOIN is the mirror of LEFT JOIN — it returns all rows from the right table and matching rows from the left. In practice, most developers rewrite RIGHT JOINs as LEFT JOINs by swapping the table order, because it is easier to reason about. Both produce identical results.

```
-- Find ALL products, including those that have never been ordered
-- Using RIGHT JOIN
SELECT p.product_name,
       p.category,
       p.price,
       COUNT(oi.item_id)  AS times_ordered
FROM order_items oi
RIGHT JOIN products p ON oi.product_id = p.product_id
GROUP BY p.product_id, p.product_name, p.category, p.price
ORDER BY times_ordered DESC;

-- Exact same result, written as LEFT JOIN (preferred)
SELECT p.product_name,
       p.category,
       p.price,
       COUNT(oi.item_id)  AS times_ordered
FROM products p
LEFT JOIN order_items oi ON p.product_id = oi.product_id
GROUP BY p.product_id, p.product_name, p.category, p.price
ORDER BY times_ordered DESC;
```

Products that have never been ordered will appear with a count of 0. This is a classic inventory question: what are we stocking that nobody buys?

2.4 FULL OUTER JOIN

A FULL OUTER JOIN returns all rows from both tables. Where there is a match, the columns from both sides are populated. Where there is no match on either side, the unmatched side comes back as NULL.

```
-- All customers AND all sessions, even if one side has no match
SELECT c.name,
       c.city,
       s.session_date,
       s.device,
       s.converted
FROM customers c
FULL OUTER JOIN user_sessions s ON c.customer_id = s.customer_id
ORDER BY c.name, s.session_date;
```

FULL OUTER JOIN is less common than INNER or LEFT, but it is useful when you want to audit two datasets against each other — finding rows that exist in one source but not the other, which is a very common data quality task.

2.5 Visualising the Four JOIN Types

```
Table A      Table B       Result

   ┌─┐  ┌─┐              INNER JOIN
   │ ●─●│  │              Only the overlap
   └─┘  └─┘              No unmatched rows

   ┌─┐  ┌─┐              LEFT JOIN
   │●●●─●│  │              Everything from A
   └─┘  └─┘              B side is NULL where no match

   ┌─┐  ┌─┐              RIGHT JOIN
   │ ●─●│●●│              Everything from B
   └─┘  └─┘              A side is NULL where no match

   ┌─┐  ┌─┐              FULL OUTER JOIN
   │●●●─●│●●│              Everything from both
   └─┘  └─┘              NULLs on whichever side has no match
```

2.6 A Word on Normalisation

Your e-commerce schema is in Third Normal Form (3NF). You do not need to memorize the formal definitions right now, but you should understand what each level means in plain terms:

First Normal Form (1NF): Every column contains a single atomic value. No repeating groups. A column called "products_bought" that stores "Mouse, Keyboard, Book" in one cell violates 1NF. Your schema satisfies this — order_items exists precisely to avoid storing multiple products in one orders column.

Second Normal Form (2NF): Every non-key column depends on the entire primary key, not just part of it. This matters when you have composite primary keys. Your schema satisfies this.

Third Normal Form (3NF): Every non-key column depends only on the primary key, not on another non-key column. A classic violation: storing both city and postal_code in customers where postal_code determines city. If you ever store something that can be derived from a non-key column, you are likely violating 3NF.

Why does this matter for data science? Because poorly normalised data creates the data quality problems you will spend hours cleaning before you can build a model. Understanding normalisation helps you design schemas where data quality problems cannot happen in the first place.

Part 3 — Lab Exercises

Create a new folder and file for this week:

```
cd ~/database-labs
mkdir lab4
nano lab4/queries.sql
```

Save every query you write into this file as you go.

Query 1 — Your first join: orders with customer names

Rewrite your Week 2 orders query so that instead of showing customer_id, it shows the customer's actual name and city.

```
SELECT o.order_id,
       c.name          AS customer_name,
       c.city,
       o.order_date,
       o.status,
       o.total_amount
FROM orders o
INNER JOIN customers c ON o.customer_id = c.customer_id
ORDER BY o.order_date DESC;
```

Before this week, you had to look up customer names separately. Now you get them in one query. Notice how much more readable the output is. This is the most common join in any analytics workflow — connecting a transaction table to a lookup table.

Query 2 — Full order receipt

Write a query that produces a complete order receipt: for every item ever purchased, show the customer's name, the order date, the product name, category, quantity, unit price, and the line total.

```
SELECT c.name                              AS customer,
       o.order_id,
       o.order_date,
       o.status,
       p.product_name,
       p.category,
       oi.quantity,
       oi.unit_price,
       oi.quantity * oi.unit_price         AS line_total
FROM order_items oi
INNER JOIN orders    o  ON oi.order_id   = o.order_id
INNER JOIN customers c  ON o.customer_id = c.customer_id
INNER JOIN products  p  ON oi.product_id = p.product_id
ORDER BY o.order_id, p.category;
```

This four-table join is the heart of any e-commerce analytics system. Every report that a business stakeholder asks for — "what did customer X buy?", "how much did category Y sell?" — starts from exactly this join. Run it and scroll through the output. You can now read the entire transaction history of your fictional store.

Query 3 — Customers who have never ordered

Use a LEFT JOIN to find customers who signed up but have never placed any order. This is the first step in any churn or re-engagement analysis.

```
SELECT c.customer_id,
       c.name,
       c.email,
       c.city,
       c.signup_date
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL
ORDER BY c.signup_date;
```

The trick here is `WHERE o.order_id IS NULL`. After a LEFT JOIN, rows from the left side that had no match come back with NULLs on the right side. Filtering for those NULLs is how you isolate the non-matching rows. This pattern — LEFT JOIN followed by WHERE right_side IS NULL — is one of the most useful patterns in SQL.

Are there any customers in your dataset who never ordered? If not, which customer has ordered the fewest times?

Query 4 — Products never ordered

Use a LEFT JOIN to find any products that have never appeared in an order_items row.

```
SELECT p.product_id,
       p.product_name,
       p.category,
       p.price,
       p.stock_qty
FROM products p
LEFT JOIN order_items oi ON p.product_id = oi.product_id
WHERE oi.item_id IS NULL
ORDER BY p.category;
```

Products that never sell are a problem: they tie up inventory budget and warehouse space. In a real business this query would run on a schedule. If there are no results on your dataset, temporarily add a product that does not appear in order_items and rerun it to confirm the query works.

Query 5 — Revenue by customer with join

Combine what you learned in Week 3 (aggregation) with this week's joins. For every customer, show their name, city, total number of orders, and total amount spent. Include customers who have never ordered, showing 0 for both counts.

```
SELECT c.name,
       c.city,
       COUNT(o.order_id)                   AS total_orders,
       COALESCE(SUM(o.total_amount), 0)    AS total_spent
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name, c.city
ORDER BY total_spent DESC;
```

You are using COUNT on o.order_id rather than COUNT*(). The difference matters: COUNT()* counts all rows, COUNT(column) counts only non-NULL values. For customers with no orders, o.order_id is NULL, so COUNT(o.order_id) correctly returns 0.

Query 6 — Best-selling products

Find the top 5 products by total units sold. Show the product name, category, price, total units sold, and total revenue generated by that product.

```
SELECT p.product_name,
       p.category,
       p.price,
       SUM(oi.quantity)                     AS units_sold,
       SUM(oi.quantity * oi.unit_price)     AS total_revenue
FROM products p
INNER JOIN order_items oi ON p.product_id = oi.product_id
ORDER BY units_sold DESC
LIMIT 5;
```

Now extend this yourself: change LIMIT 5 to show all products, and add a LEFT JOIN instead of INNER JOIN so products with zero sales also appear. What do you need to change about SUM(oi.quantity) to handle the NULLs?

Query 7 — City-level sales report

For each city, show how many customers are based there, how many orders those customers have placed, and the total revenue from those orders. Only show cities where at least one order has been placed.

```
SELECT c.city,
       COUNT(DISTINCT c.customer_id)        AS num_customers,
       COUNT(DISTINCT o.order_id)            AS num_orders,
       SUM(o.total_amount)                   AS total_revenue,
       ROUND(AVG(o.total_amount), 2)         AS avg_order_value
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.city
ORDER BY total_revenue DESC;
```

Notice COUNT(DISTINCT c.customer_id) rather than COUNT(c.customer_id). After the join, a customer with three orders would appear three times — using DISTINCT ensures you count each customer once. This is a subtle but important difference and a very common source of incorrect results in real analytics work.

Query 8 — Order details with status filter

Using the full four-table join from Query 2 as your base, now filter to show only delivered orders from 2025. Group by customer and show each customer's total spend and the total number of distinct products they purchased.

```
SELECT c.name                               AS customer,
       c.city,
       COUNT(DISTINCT p.product_id)          AS distinct_products_bought,
       SUM(oi.quantity)                      AS total_items,
       SUM(oi.quantity * oi.unit_price)      AS total_spent
FROM order_items oi
INNER JOIN orders    o  ON oi.order_id    = o.order_id
INNER JOIN customers c  ON o.customer_id  = c.customer_id
INNER JOIN products  p  ON oi.product_id  = p.product_id
WHERE o.status      = 'delivered'
  AND o.order_date >= '2025-01-01'
GROUP BY c.customer_id, c.name, c.city
ORDER BY total_spent DESC;
```

This combines joins, WHERE filters, GROUP BY, and aggregate functions in one query. By this point in the course you have used every fundamental SQL tool. The rest of the semester is about combining and applying them.

Query 9 — Session and purchase behaviour together

Revisit the session-to-purchase analysis from Week 3, but this time use a proper join structure instead of separate CTEs. For every customer who has at least one session, show their name, city, number of sessions, whether they have ever purchased, and if so their total spend.

```
SELECT c.name,
       c.city,
       COUNT(DISTINCT s.session_id)          AS total_sessions,
       COUNT(DISTINCT o.order_id)             AS total_orders,
       COALESCE(SUM(DISTINCT o.total_amount), 0) AS total_spent,
       CASE
           WHEN COUNT(DISTINCT o.order_id) > 0 THEN 'Converted'
           ELSE 'Not Converted'
       END                                     AS customer_status
FROM customers c
INNER JOIN user_sessions s ON c.customer_id = s.customer_id
LEFT JOIN  orders        o ON c.customer_id = o.customer_id
                            AND o.status = 'delivered'
GROUP BY c.customer_id, c.name, c.city
ORDER BY total_sessions DESC;
```

Notice the join condition `AND o.status = 'delivered'` is written inside the ON clause rather than in WHERE. This is intentional — putting it in WHERE would turn the LEFT JOIN into an effective INNER JOIN. When you want to filter one side of a LEFT JOIN without eliminating non-matching rows, the condition goes in ON.

Query 10 — FULL OUTER JOIN: data completeness audit

Write a query using FULL OUTER JOIN to identify any mismatches between the customers table and the user_sessions table. Find customers who have no session data, and any orphaned session records with no matching customer.

```
SELECT c.customer_id                        AS customer_table_id,
       c.name,
       s.customer_id                        AS session_table_cust_id,
       COUNT(s.session_id)                  AS session_count,
       CASE
           WHEN c.customer_id IS NULL THEN 'Orphaned session — no customer
record'
           WHEN s.customer_id IS NULL THEN 'Customer with no sessions'
           ELSE 'Matched'
       END                                  AS audit_status
FROM customers c
FULL OUTER JOIN user_sessions s ON c.customer_id = s.customer_id
GROUP BY c.customer_id, c.name, s.customer_id
ORDER BY audit_status, c.name;
```

In real data work this kind of audit query runs before any analysis. If you find orphaned records, they indicate a data pipeline problem. If you find customers with no sessions, those might be legitimate or might indicate a tracking gap. Either way, you need to know before you build a model.

Save and Commit

```
cd ~/database-labs
git add lab4/queries.sql
git commit -m "Lab 4: Completed JOIN exercises — INNER, LEFT, RIGHT, FULL OUTER"
git push origin main
```

Part 4 — Performance Awareness

JOINs are where query performance gets genuinely interesting.

```
EXPLAIN ANALYZE
SELECT c.name, o.order_id, o.total_amount
FROM orders o
INNER JOIN customers c ON o.customer_id = c.customer_id
WHERE o.status = 'delivered';
```

Look for **Hash Join** or **Nested Loop** in the output. These are the two main strategies PostgreSQL uses to execute joins.

A **Nested Loop** join works like two nested for-loops: for each row in table A, scan table B for a match. On small tables this is fine. On large tables it becomes $O(n^2)$ — extremely slow.

A **Hash Join** builds a hash table from one side, then probes it for each row from the other side. Much faster on large datasets, but requires memory to build the hash table.

On your small dataset PostgreSQL will probably choose Nested Loop because it is faster for small data. On a table with millions of rows it would switch to Hash Join automatically. EXPLAIN ANALYZE lets you see what it chose.

For your deliverable, run EXPLAIN ANALYZE on Query 2 (the four-table join). Note every operation type that appears and write one sentence explaining what each one does.

Part 5 — AI Prompting for This Week

JOINs are where students most commonly get wrong results without getting an error. The query runs, returns rows, but the numbers are subtly wrong — usually because of a missing DISTINCT, a filter in the wrong place, or choosing INNER instead of LEFT. AI is very useful here for debugging logic rather than syntax.

Pattern 1 — Understanding why your row count looks wrong

```
I wrote this SQL query in PostgreSQL and the row count looks too high.
I expected around 20 rows but I am getting 47.

SELECT c.name, COUNT(o.order_id) AS total_orders
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.name;

My tables:
- customers (customer_id, name, email, city, signup_date) — 20 rows
- orders (order_id, customer_id, order_date, status, total_amount) — 30 rows

Some customers have placed multiple orders.

Please:
1. Explain why the row count might look unexpected
2. Check whether my GROUP BY is correct for this goal
3. Explain when to use COUNT(*) vs COUNT(column) vs COUNT(DISTINCT column)
4. What would change if I used LEFT JOIN instead of INNER JOIN?
```

Pattern 2 — Choosing the right join type

```
I am trying to write a SQL query but I am not sure whether to use
INNER JOIN or LEFT JOIN.

My goal: Show all customers with a count of their orders. Customers
who have never ordered should appear with a count of 0, not be excluded.

My tables:
- customers (customer_id, name, city, signup_date)
- orders (order_id, customer_id, order_date, status, total_amount)

Please:
1. Explain the difference between INNER and LEFT JOIN for this specific goal
2. Show both versions and explain what each returns differently
3. Explain the WHERE o.order_id IS NULL pattern and when to use it
4. What is the most common mistake people make when switching
   between INNER and LEFT JOIN?
```

Pattern 3 — Debugging a multi-table join producing duplicates

```
My SQL query is producing duplicate rows and I cannot figure out why.

SELECT c.name, SUM(o.total_amount) AS total_spent
FROM customers c
INNER JOIN orders o      ON c.customer_id = o.customer_id
INNER JOIN user_sessions s ON c.customer_id = s.customer_id
GROUP BY c.name;

The total_spent numbers seem much too large compared to what I expect.

My tables:
- customers: 20 rows
- orders: 30 rows
- user_sessions: 56 rows

Please:
1. Explain exactly why this query produces inflated numbers
2. What is a "join explosion" or "fan-out" and is that happening here?
3. Show me two correct approaches to fix this
4. Explain the general rule for when joining multiple tables can
   cause unexpected duplication
```

Part 6 — Deliverables

Submit a single PDF named `Lab04_FirstNameLastName_RollNumber.pdf` to Google Classroom by the announced deadline. Late penalty: 20% per day.

Section 1 — 10 Queries (35 points)

For each query: formatted SQL code, screenshot of results, and two to three sentences of business-level interpretation. By now the bar is high — explain what the result means, not just what the query does.

Section 2 — ER Diagram (20 points)

Draw the ER diagram for your complete schema: customers, products, orders, order_items, and user_sessions. You can draw it by hand and photograph it, or use a free tool like dbdiagram.io or draw.io. It must show: all five tables, their columns, their primary keys (underlined or marked PK), and all foreign key relationships with directional arrows. Label the relationship type (one-to-many) on each arrow.

Section 3 — Normalisation Reflection (15 points)

Choose one of the following and write a 200-word response. Either: (a) Describe a real-world scenario where you have seen or can imagine data stored in an unnormalised way, explain what problems that causes, and show how you would normalise it into at least two tables. Or (b) Look at the user_sessions table and argue whether it is in 3NF. If you think it violates any normal form, explain the violation and propose a fix.

Section 4 — AI Learning Log (20 points)

Minimum 3 entries. At least one must involve debugging a join that produced unexpected results — show the wrong query, the AI interaction, what you learned, and the corrected version.

Section 5 — Reflection (10 points)

150 words minimum. What is the difference between INNER JOIN and LEFT JOIN in your own words — when would you use each? What is the most surprising thing you discovered by joining the tables together that you could not see looking at them individually?

Quick Reference

```
-- ── JOIN skeleton ──────────────────────────────────────────
SELECT a.col1, b.col2
FROM   table_a a              -- left table
JOIN   table_b b              -- right table (INNER by default)
    ON a.id = b.a_id          -- join condition
WHERE  ...                    -- row filter (applied after join)
ORDER BY ...;

-- ── JOIN types ─────────────────────────────────────────────
INNER JOIN  -- only matching rows from both sides
LEFT JOIN   -- all rows from left, NULLs where no right match
RIGHT JOIN  -- all rows from right, NULLs where no left match
FULL OUTER JOIN -- all rows from both sides, NULLs where no match

-- ── Find rows with no match (anti-join) ────────────────────
SELECT a.*
FROM   table_a a
LEFT JOIN table_b b ON a.id = b.a_id
WHERE  b.id IS NULL;          -- rows in A with no match in B

-- ── Multi-table join ───────────────────────────────────────
SELECT c.name, o.order_id, p.product_name, oi.quantity
FROM   order_items  oi
JOIN   orders       o   ON oi.order_id   = o.order_id
JOIN   customers    c   ON o.customer_id = c.customer_id
JOIN   products     p   ON oi.product_id = p.product_id;

-- ── COUNT with joins — important distinctions ──────────────
COUNT(*)                 -- every row, including duplicates
COUNT(o.order_id)        -- non-NULL values only
COUNT(DISTINCT o.order_id) -- unique values only

-- ── Filter inside ON vs WHERE ──────────────────────────────
-- This keeps all left rows (TRUE left join behaviour)
LEFT JOIN orders o ON c.customer_id = o.customer_id
                   AND o.status = 'delivered'

-- This eliminates non-matching left rows (acts like INNER JOIN)
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.status = 'delivered'
```